Project 1: Google in one day

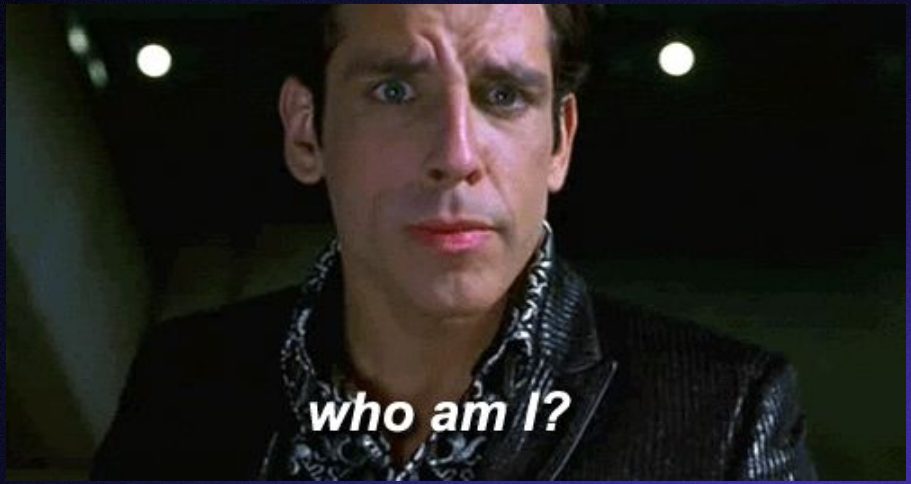
Using AI Coding tools

March 9, 2026

David Dogukan Erenel

Prepared for Istanbul Technical University - AI Aided Computer Engineering Course

non proprietary code or data


who am I?

David Dogukan Erenel: Who am I?

Career

Principal Software Engineer at Gusto

~7 yrs at Meta, Menlo Park, CA

~3 yrs at IBM, Austin, TX

~2 yrs in Startup, Miami, FL

~3.5 yrs Startup co-founder, Istanbul / Türkiye

Education

Lead Program, Stanford University Graduate School of Business, 2023

MSc, Bogazici University, 2011

BS, Istanbul Technical University, 2008

Personal

2 kids

Living in NJ

Family & Friends, Growth, Transparency and Humor

I know who I am

You can ask me anything!
FANMIO

Q/A

https://app.sli.do/event/fwRS1JJyChJL9G3mT1FzKV

Project 1: **Google in a day**

The Challenge

Build a functional web crawler and real-time search engine from scratch.

The Shift

You are no longer just a "coder", you are a System Architect steering AI agents (Cursor, Claude, VS Code) to build complex, concurrent software.

The Goal

Demonstrate architectural sensibility, concurrency management, and "Human-in-the-Loop" verification.

Technical Requirements (**Indexer**)

Recursive Crawling
Initiate from an origin URL to a maximum depth *k*.

Uniqueness
Implement a "Visited" set to ensure no page is crawled twice.

Back Pressure
The system must manage its own load (e.g., max rate of work or queue depth).

Native Focus / Constraint
You must prompt the AI to use language-native functionality (e.g., net/http or urllib) rather than high-level libraries like Scrapy or Beautiful Soup.

Technical Requirements (**Searcher**)

Query Engine
Return a list of triples: (relevant_url, origin_url, depth).

Live Indexing
Search must be able to run while the indexer is active.

Concurrency
Use AI to design thread-safe data structures (e.g., Mutexes, Channels, or Concurrent Maps) to prevent data corruption.

Relevancy
Define a simple heuristic (e.g., keyword frequency or title matching) to rank results.

The AI-Augmented Workflow (cheatsheet)

Phase 1 (PRD)
Use Claude / ChatGPT / Gemini to expand this brief into a formal Product Requirement Document.

Phase 2 (Design)
Use Cursor/Composer to map out the system architecture before writing logic.

Phase 3 (Prompting)
Use User Profile Prompting (e.g., .cursorrules) to set coding standards and constraints.

Phase 4 (Iterative Coding)
Build the core crawler first, then the search interface, then the UI.

System Visibility & UI

UI/CLI Requirements

You must build a dashboard to view the state of the system in real-time.

Metrics to Track:

- Current Indexing Progress (URLs processed vs. queued).
- Current Queue Depth.
- Back-pressure/Throttling status.

Persistence (Bonus)

Design the system so it can be resumed after an interruption without restarting the crawl.

What Success Looks Like

Functionality (40%): Does it crawl accurately and search concurrently?

Architectural Sensibility (40%): How well did you handle back pressure and thread safety?

AI Stewardship (20%): Can you explain the AI-generated code and justify the design choices?

Final Output:

- Final-4: **Code, Readme, PRD, Recommendation**
- Working Codebase (Github public repository) including:
  - Readme (**readme.md**)
  - PRD as markdown (**product_prd.md**)
  - A 2-paragraph Production Roadmap:
    - Recommendations for deploying this into a high-scale production environment. (**recommendation.md**)

Q/A

https://app.sli.do/event/fwRS1JJyChJL9G3mT1FzKV